Sachai Check — Urdu Fake News Detection: Pipeline Design Notes

Project: Sachai Check (Multi-Script Fake News & Audio Deepfake Detection for Urdu)
Team: Muhammad Abdullah Haroon (23L-0734), Ikhlas Ahmad (23L-0638), Zayan Amjad (23L-0721) **Supervisor:** Miss Sukhan Amir, FAST-NUCES **Scope of this doc:** design rationale for the Urdu fake-news text pipeline (the audio deepfake and full backend/frontend build stay in the runbook as-is)

---

1. The Problem, As Originally Raised

Starting the Urdu fake news detection part first, with these open concerns:

- Want a **large corpus** covering multiple domains, not just politics, with real evaluation of whether the model is actually good — not just a high headline accuracy.
- Already hit a **spurious-correlation failure once**: a length-based cue ("fake news tends to be longer") that scored ~75% on one dataset collapsed to near-chance (~0.7%... effectively random) when evaluated on a different dataset. Worried a bigger combined corpus makes this worse, not better, if not handled carefully.
- Want to beat existing Urdu fake-news systems, not just replicate one — the differentiator being **coverage and honesty**, not just another classifier.
- **Out-of-scope input**: what happens if someone submits Urdu text about a topic outside the training distribution — e.g. an Urdu-language claim about Argentina, Mexico, England, or an Indian movie, rather than Pakistan-centric news? How does the system know it's out of its depth instead of guessing?
- **Recency / up-to-date facts**: how does the system verify a claim like "Spain won the 2026 World Cup" when that event happened after any static training cutoff?
- **Contradictions over time**: two countries at war, later signing a peace treaty — old "true" statements become false, new ones become true. Will a frozen classifier get confused by this, and how should the system handle it?
- **Is it really this simple** — gather data, merge into one big dataset, train, test? Or is there more structure needed?

- **Operational question**: how to keep the system "alive" and current without retraining the model every single day, which isn't feasible for an FYP team.

---

2. Core Design Principle

A trained classifier should never be the system's source of truth for facts. It should only judge linguistic/stylistic authenticity signals — is this written the way real vs. fabricated Urdu news is typically written. Anything that depends on "is this true right now in the real world" has to come from retrieval against live sources, not from frozen model weights.

This single principle resolves most of the open questions above:

Concern	Handled by
Length/style overfitting	Classifier + debiasing + cross-dataset eval
Domain generalization	Multi-domain corpus + per-domain evaluation gate
Out-of-scope input (non-Pakistan Urdu news)	Relevance/OOD check before classification
Recency (2026 World Cup, breaking news)	LLM judge layer with live retrieval, not classifier
Contradictions over time (war → peace)	Retrieval naturally returns *current* state, no need to "reconcile" old vs new inside the model
Keeping the system current	Retrieval refreshed per-query; classifier retrained on a slow cadence, not daily

So the system is **two cooperating layers**, not one model:

1. **Stylistic/authenticity classifier** (transformer, fine-tuned once, updated occasionally) — answers "does this read like fabricated Urdu news."
2. **Retrieval-augmented LLM judge** (Gemini + live search, runs fresh every request) — answers "what does current, real-world evidence say about this specific claim," and produces the explanation, supporting/contradicting claims, and a confidence score.

The final verdict shown to the user is a **combination** of both signals, not either one alone — and the judge layer should be allowed to override or flag the classifier when live evidence disagrees with it (e.g. classifier says "reads like real news" but retrieval finds no

corroborating source at all).

---

3. Handling the Overfitting / Length-Bias Problem

The 75% → near-chance collapse happened because the model learned "long text = fake" instead of "fake = fake" — the classic spurious-correlation trap, and a known issue in the Bend-the-Truth line of work (the FIRE-2021 shared task organizers explicitly instructed their journalists to *match* real/fake article lengths for this exact reason).

Mitigations, in order of importance:

- **Balance length distribution within each class, per domain** — not just balance real vs. fake counts overall. Check this explicitly in EDA, split by class, before training.
- **Cross-dataset evaluation as a standing protocol, not a one-off check.** Train on dataset A + B, test on a fully held-out dataset C. This is the same SS/ST robustness methodology already used in the Urdu/Roman-Urdu script-robustness paper — reuse it here directly.
- **Per-domain evaluation as a gate, not just an overall F1 number.** If one domain's F1 collapses, that's a real signal to fix before moving forward, not something to average away.
- **Prefer datasets where fake examples were deliberately length-matched to real ones** (Bend-the-Truth is explicitly built this way) over datasets where fake news was scraped opportunistically and may carry incidental length/format artifacts.

---

4. Handling Out-of-Scope / Off-Domain Input

For Urdu text about topics outside the corpus's effective coverage (e.g. non-Pakistan/South-Asia claims):

- Add a **relevance/OOD check before the classifier fires** — either a lightweight secondary classifier, or a direct prompt to the LLM judge: "is this claim about Pakistan/South Asia, or a topic this system wasn't built to verify?"
- If flagged out-of-scope, the system should **surface a caveat instead of a confident verdict** ("outside training domain — treat this result with lower confidence") rather than silently guessing.
- Document this as a known boundary in the report's limitations section — an acknowledged scope boundary is a strength in a panel review, not a weakness.

5. Handling Recency and Contradictions Over Time

This is the part that a static classifier genuinely cannot do, and it's the reason the LLM judge layer exists:

- Give the judge model **tool access to live web/news search** before it answers. A claim like "Spain won the 2026 World Cup" gets checked against current sources at query time, not against whatever the model happened to know at training time.
- For contradiction-over-time cases (a war that became a peace treaty), retrieval solves this **for free** — you're always pulling the current state of the world, so there's no need for the model to "remember" and reconcile an old fact against a new one internally.
- The judge's output should include **source recency** (date of the retrieved evidence) alongside the explanation, so the user can see *when* the supporting/contradicting evidence is from — this also makes a stronger report section than a black-box verdict.
- This is conceptually the same approach used by real-world fact-checking systems (e.g. Full Fact, ClaimBuster) — retrieval-augmented verification, not a frozen classifier claiming to know the news.

---

6. Keeping the System Updated Without Retraining Daily

Retraining the transformer every day is neither feasible for an FYP team nor actually necessary, because **the two layers update on different clocks**:

Layer	What it needs to stay "current"	Update cadence
Classifier (stylistic/authenticity)	Learns *how* fake Urdu news is typically written — a pattern that changes slowly	Train once per major sprint milestone (e.g. after Sprint 3, then maybe once more before Sprint 12 if new data was added). No need to retrain per new event.
LLM judge + retrieval	Needs to know *what's currently true*	Runs a fresh live search **on every single query** — this is what makes the system feel "up to date" without ever touching model weights
	Needs to be running	Load both models once at startup (already planned in SC-29); never reload per

Server	and responsive	request

So "keeping it updated every day" is **not a training problem at all** — it's a retrieval problem, and it's already solved by having the judge layer call a live search API per request. The only thing that runs on a slower, deliberate cadence is:

- **Optional periodic corpus refresh**: every few weeks, add newly-verified real/fake examples (the way SC-7's 45 manually-verified claims were collected) and re-fine-tune the classifier if you have time before the deadline — but this is a nice-to-have polish step, not a daily requirement.
- **No daily retraining pipeline needs to be built at all** for this FYP — mentioning this as a deliberate design decision (not an oversight) is a good line for the "why we did it this way" section of the report.

---

7. Datasets and Domains to Use

Urdu fake-news text datasets

Dataset	Domains covered	Size	Notes
Bend-the-Truth (Amjad, Sidorov et al., 2020)	Technology, education, business, sports, politics, entertainment (6 domains in the original release)	900 articles (500 real / 400 fake)	The standard benchmark for Urdu fake news; fake articles were written by professional journalists and **deliberately length-matched** to real ones — directly mitigates your length-bias problem. Real news sourced from BBC Urdu, CNN Urdu, Express News, Jang, Nawa-i-Waqt, etc.
UrduFake@FIRE2021 shared-task dataset	Business, health, sports, showbiz (entertainment), technology (5	~2,300+ articles (expanded from Bend-the-Truth for the shared	Same lineage as Bend-the-Truth, explicitly built with length-matching instructions to journalists — good second source for cross-dataset

	domains)	task)	evaluation (train on one, test on the other).
mirfan899/Urdu — `real_fake_news` + `urdu-news-dataset-1M` (GitHub)	Business & economics, entertainment, science & technology, sports	1M+ real news headlines/articles (for volume/domain balance), smaller labelled real/fake subset	Useful mainly to bulk up the **real-news volume and domain diversity**, since Bend-the-Truth alone is small; use cautiously and re-verify labels rather than trusting them blindly.

There is **no established Roman Urdu fake-news corpus** — this matches what the runbook already assumes (SC-9): build a small manually-verified Roman Urdu test set (~20 examples) rather than expecting to find one pre-made. Treat Roman Urdu as a **transliteration + robustness test set**, not a training source.

Urdu audio deepfake dataset

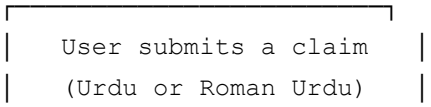
Dataset	Details
CSALT-LUMS Urdu Deepfake Audio Dataset (Munir, Sajjad, Raza et al., ACL 2024 Findings — "Deepfake Defense")	Bonafide (real) audio plus two TTS spoofing methods (Tacotron, VITS). Built with attention to phonemic coverage/balance, benchmarked against AASIST-L. Available on GitHub and Hugging Face. This is the dataset the runbook already points to (SC-10).

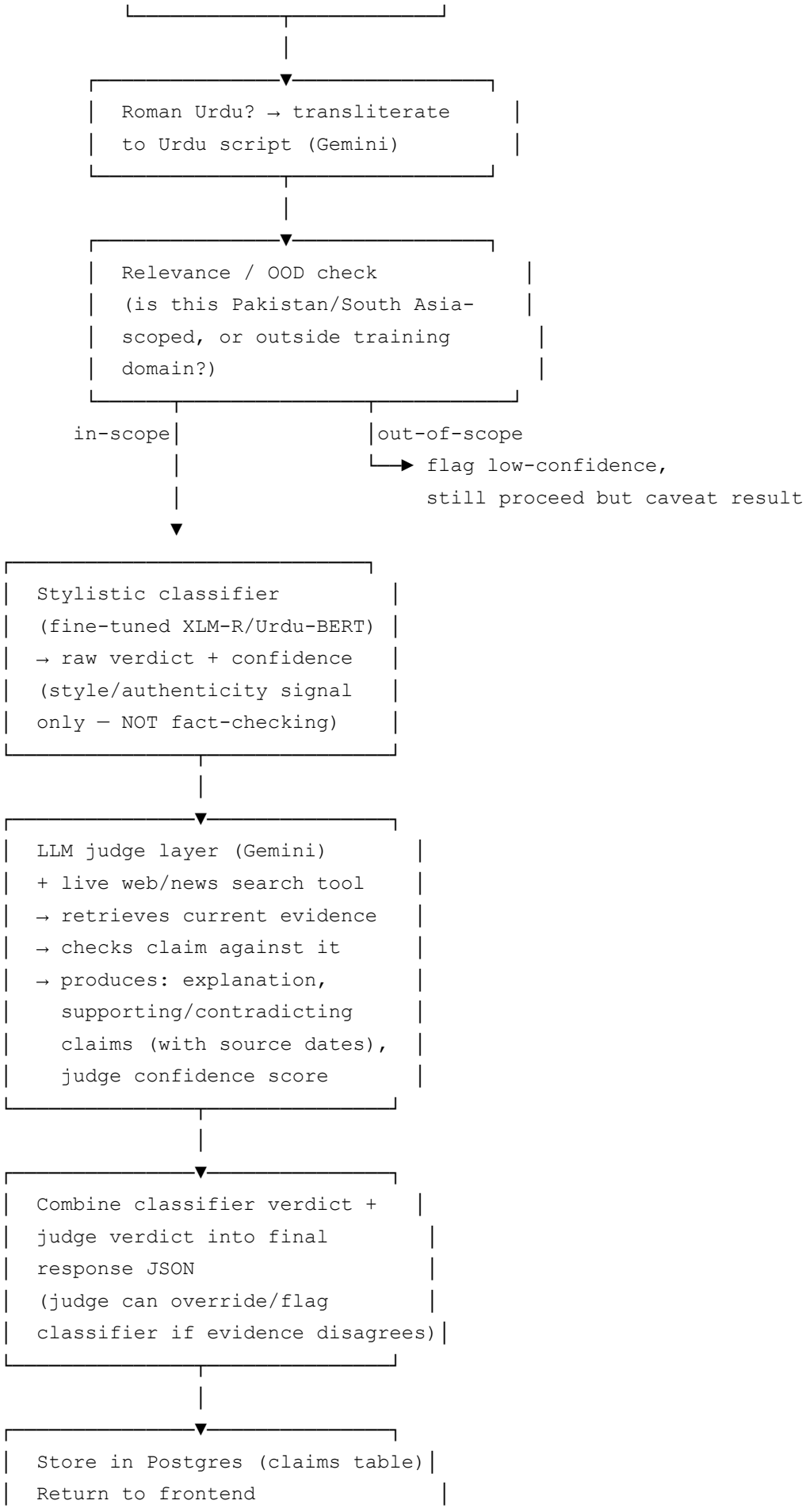
Manually-collected supplementary data

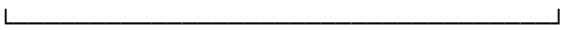
- **45 manually-verified real-world claims** (15 per team member, per SC-7) — the most valuable data you'll have for testing genuine recency/generalization, since it's not from any existing corpus and can be chosen specifically to include recent, cross-domain, and non-Pakistan-context examples to stress-test the OOD and recency handling described above.

---

8. Pipeline Structure (End-to-End)




Roman Urdu? → transliterate to Urdu script (Gemini)
Relevance / OOD check (is this Pakistan/South Asia-scoped, or outside training domain?)
in-scope
out-of-scope
flag low-confidence, still proceed but caveat result
Stylistic classifier (fine-tuned XLM-R/Urdu-BERT) → raw verdict + confidence (style/authenticity signal only — NOT fact-checking)
LLM judge layer (Gemini) + live web/news search tool → retrieves current evidence → checks claim against it → produces: explanation, supporting/contradicting claims (with source dates), judge confidence score
Combine classifier verdict + judge verdict into final response JSON (judge can override/flag classifier if evidence disagrees)
Store in Postgres (claims table)
Return to frontend



Audio path runs in parallel: audio → preprocess (resample/trim) → mel-spectrogram → CNN classifier → same LLM judge layer for explanation generation (the judge layer is shared infrastructure between both pipelines).

What runs once vs. per-request:

- Loaded once at server startup: transformer classifier, CNN classifier.
- Run fresh on every request: OOD check, live search/retrieval, judge reasoning.
- Run on a slow, deliberate cadence (not daily): classifier fine-tuning, if the corpus is expanded later.

---

9. Report-Writing Note

This split — classifier for style, retrieval-augmented judge for facts, explicit OOD/scope boundary, cross-dataset validation for bias — makes a stronger **methodology section** than "we merged datasets and trained a model," because it shows the design was driven by a real, documented failure mode (the length-bias collapse) rather than assumed to work. That failure and its fix is worth including directly in the report's methodology and limitations sections as evidence of rigor.